CloudSentinel AI: A Context-Aware Framework for Cloud Misconfiguration Risk Assessment and Candidate Attack-Path Analysis

Harshith

***Abstract*—Cloud Security Posture Management (CSPM) tools commonly report findings at the individual-resource level, which can obscure relationships among cloud assets and complicate remediation prioritisation. This paper proposes CloudSentinel AI, an AWS-focused framework for context-aware cloud misconfiguration assessment. The design collects selected configuration metadata, models assets and relationships in a knowledge graph, evaluates documented security rules, and derives candidate attack paths subject to explicit preconditions. A transparent contextual scoring heuristic is proposed to prioritise findings using asset criticality, exposure, identity permissions, data sensitivity, and graph context. An LLM-assisted reporting component is constrained to evidence-bounded, human-reviewed explanations and remediation recommendations. The paper also defines a controlled AWS evaluation protocol based on labelled scenarios. A tagged, isolated AWS baseline and a private S5 workload have been deployed, configuration-verified, and assessed by read-only evidence-collection passes. The passes detected a bounded security-group configuration finding and a temporary bucket-scoped IAM wildcard permission, while rejecting candidate paths whose reachability preconditions were absent. A private S5 instance-profile relationship to a synthetic S3 object and metadata-only secret was observed, but no candidate path was asserted. An account-level S3 safeguard blocked attachment of the planned empty-bucket public policy. A dedicated S4 CloudTrail test trail was also observed in the disabled state without altering an existing trail. S6 and comparative scenario-level experiments remain unexecuted. This work therefore does not claim completed experimental validation or general production effectiveness.**

***Index Terms*—cloud security, cloud misconfiguration, AWS, knowledge graph, attack-path analysis, risk prioritisation, LLM-assisted security reporting**

I. Introduction

Cloud computing provides on-demand access to configurable computing resources [1], but security posture depends on the interaction of identities, network controls, storage policies, logging, and service relationships. An isolated configuration finding does not necessarily convey its operational context. For example, the priority of an exposed security group depends on the associated workload, reachable assets, privileges, and data sensitivity.

CloudSentinel AI is proposed as an AWS-focused research framework that combines configuration assessment with relationship-aware reasoning. The study follows a design-science approach in which the framework is the research artifact and the evaluation protocol tests its bounded claims [2]. Its intended contributions are: (i) an AWS resource-and-relationship model for a security knowledge graph, (ii) a candidate attack-path method that records explicit preconditions, (iii) a transparent contextual prioritisation heuristic, and (iv) an evidence-bounded LLM-assisted reporting workflow. The framework is a design proposal. Any claim of improved detection, prioritisation, or analyst effectiveness requires the controlled evaluation described in this paper.

II. Background and Related Work

Knowledge graphs provide a flexible representation for entities and their relationships [3]. Attack-graph research has long used graph models to reason about possible sequences of security-relevant states [4], [5]. More recent cloud-focused work also illustrates the usefulness of graph reasoning for identifying critical paths in cloud tenants [6]. These methods motivate relationship-aware analysis but do not remove the need to state attacker assumptions and validate path preconditions.

Cloud misconfiguration analysis is also important for identity controls. For example, anomalous AWS IAM policies can be identified through policy analysis [7]. Explainability is an active cybersecurity research area [8]; however, generated natural-language text alone is not explainability. LLM-assisted output must be grounded in deterministic evidence and reviewed by a human, especially when it contains remediation advice [9]. CloudSentinel does not claim to replace these approaches. Its proposed research gap is the explicit combination of AWS configuration findings, evidence-backed candidate-path conditions, transparent contextual prioritisation, and human-reviewed LLM reporting within one reproducible evaluation protocol.

III. Proposed Framework

A. Scope and Threat Model

The initial scope is an AWS account containing EC2, S3, IAM, VPC/security-group, RDS, Secrets Manager, and CloudTrail configuration metadata. The framework assumes read-only collection credentials and evaluates only the resources made visible by those credentials. The threat model begins with an external or authenticated actor at a documented entry point and considers only transitions supported by collected configuration evidence. It does not assert an initial compromise, credential theft, exploit availability, or successful lateral movement unless those preconditions are explicitly represented and independently reviewed.

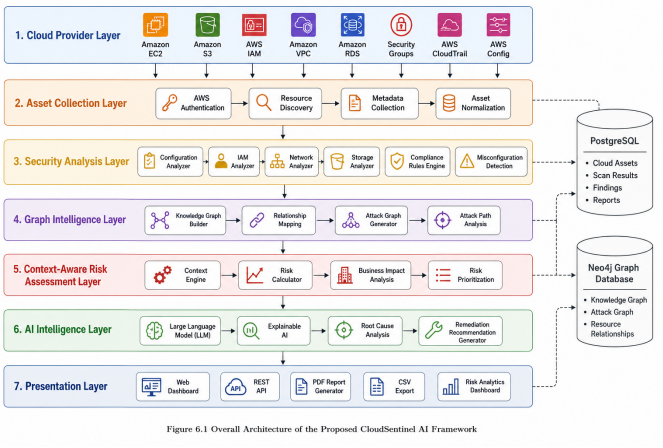


Figure 6.1 Overall Architecture of the Proposed CloudSentinel AI Framework

Fig. 1. High-level CloudSentinel AI architecture. The figure illustrates the proposed pipeline; it is not an implementation-performance result.

TABLE I
MINIMUM PROPOSED GRAPH SCHEMA

Element	Required attributes or semantics
Nodes	AWS resource, identity, policy, security finding, network boundary, or data asset; each has an account, region, source timestamp, and evidence identifier.
Edges	Typed relationship such as *attached-to*, *permits*, *can-reach*, *can-assume*, *stores*, or *has-finding*; every edge identifies its source metadata.
Findings	Rule identifier, affected node, observed evidence, severity mapping, and collection time.
Path record	Ordered edges, entry condition, required preconditions, target, supporting findings, and review status.

B. Architecture and Graph Schema

The proposed pipeline contains five stages. First, an asset collector retrieves versioned AWS metadata. Second, a configuration analyzer evaluates versioned rules derived from documented security guidance. Third, a graph builder represents assets and supported relationships as typed nodes and edges. Fourth, a candidate-path engine searches the graph under the threat model. Finally, a reporting component presents findings, path evidence, prioritisation rationale, and advisory remediation guidance. Table I defines the minimum graph schema.

C. Candidate-Path Analysis and Prioritisation

A graph edge represents a documented relationship or security-relevant condition; it does not independently prove exploitability. A candidate path is a bounded, loop-free sequence from a documented entry condition to a target asset. The engine will enumerate at most k paths with a maximum depth d, reject paths with a missing required precondition, and retain evidence for every edge. It therefore performs bounded candidate-path discovery rather than claiming exhaustive enumeration of all attack paths.

TABLE II
CLAIM-TO-EVIDENCE MAP FOR EVALUATION

Claim	Required evaluation evidence and limitation
Finding detection	Scenario ground truth, normalised scan output, and finding-level precision/recall; limited to selected rules and scenarios.
Candidate-path quality	Expected-path catalogue, edge evidence, and independent path review; graph connectivity alone is insufficient.
Prioritisation	Predefined reference ranking and sensitivity analysis; R is a heuristic, not compromise probability.
LLM-assisted reporting	Human rubric for evidence consistency, accuracy, clarity, and actionability; output remains advisory.

The proposed prioritisation score is a transparent heuristic:

$$R = \alpha S + \beta C + \gamma P + \delta E, \tag{1}$$

where all components are normalised to $[0, 1]$: S is documented finding severity, C is asset criticality, P is evidence-supported candidate-path context, and E is externally reachable exposure. The non-negative weights sum to one and will be fixed before evaluation. Exposure is excluded from C to avoid double counting. R is a prioritisation score, not an estimated probability of compromise; sensitivity analysis will report how ranking changes under alternative weights.

D. LLM-Assisted Reporting Contract

The reporting component receives only approved, evidence-bounded findings, graph relationships, and remediation context. Each generated report must distinguish observed evidence from assumptions, identify missing preconditions, link each recommendation to relevant findings, and state that human review is required. Prompt engineering cannot guarantee accuracy or prevent hallucinations; the component is advisory and will be evaluated for evidence consistency, technical accuracy, clarity, and actionability. It must not automatically execute remediation or receive unapproved sensitive data.

IV. CONTROLLED EVALUATION DESIGN

The proposed evaluation uses an isolated, authorised, non-production AWS account. All resources will be tagged, synthetic or non-sensitive data will be used, least-privilege roles and budget alarms will be configured, and every scenario will have a verified cleanup record. Intentionally weak configurations will remain within the defined laboratory boundary and will not be exposed to the public Internet.

A. Ground Truth and Units of Analysis

The evaluation distinguishes three units. A *finding* is one rule violation attached to a resource. A *candidate path* is an ordered graph sequence with documented preconditions. A *consolidated alert* groups related findings for a shared path or asset context. Before each run, a scenario catalogue will record the intentional configuration state, expected findings, negative controls, expected and forbidden candidate paths,

TABLE III
CONTROLLED AWS SCENARIO CATALOGUE

ID	Scenario	Evaluation focus
S1	Public S3 storage exposure	Finding detection, evidence capture, and remediation guidance for a test-only bucket.
S2	Overly permissive IAM policy	Detection of test-principal permissions and supported identity relationships.
S3	Insecure network configuration	Detection of a laboratory security-group condition and documented reachability preconditions.
S4	Disabled CloudTrail logging	Detection of an audit-logging condition and restoration recommendation.
S5	Multi-stage candidate path	Evidence-backed relationship reasoning across test EC2, IAM, Secrets Manager, and S3 resources.
S6	SSRF-to-IAM credential-access candidate path	Identification of configuration and identity preconditions in an instrumented, isolated test service; no public exposure or production credentials.

severity rationale, and permitted remediation guidance. This catalogue is the ground truth; it must be frozen before scanning and independently reviewed.

B. Metrics and Path Review

Finding-level precision, recall, F1, and false-positive rate are computed only against the labelled scenario finding set:

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP+FP}, \\ \text{Recall} &= \frac{TP}{TP+FN}, \\ F1 &= \frac{2TP}{2TP+FP+FN}. \end{aligned} \tag{2}$$

The false-positive rate is $FP/(FP+TN)$ when negative controls define TN. A candidate path is an *exact match* when its entry condition, ordered evidence-backed edges, target, and required preconditions match the catalogue; it is a *partial match* when it represents a valid subset or superset with the difference stated; otherwise it is *invalid*. Prioritisation quality will be measured against a pre-specified reference ranking, while alert reduction will be reported as consolidated alerts divided by normalised baseline findings for the same scenario. These measures must not be aggregated across units of analysis.

C. Execution and Baseline Protocol

For every scenario, the study will archive the pre-scan configuration snapshot, resource inventory, ground-truth catalogue, tool version, enabled checks, account scope, permissions, execution time, raw output, graph export, candidate-path evidence, and cleanup record. CloudSentinel and selected baselines will use the same account scope, permissions, configuration, and execution date. Findings will be normalised to the defined finding unit before comparison. Each timing measurement will use at least three non-warm-up repetitions and report the timing boundary, median, range, failures, host specification, AWS region, and any external API latency. A local in-memory fixture runner validates catalogue handling and attack-graph integration before cloud deployment; its outputs are regression checks and are excluded from all cloud-performance and accuracy metrics. Capability matrices based on vendor documentation will be labelled as documentation-based feature comparisons, not benchmark performance results.

D. LLM-Report Review

Each LLM-assisted report will be evaluated by at least two independent reviewers using a pre-defined rubric. The rubric scores: (i) evidence consistency, (ii) technical accuracy, (iii) clarity, and (iv) actionability. Reviewers will mark unsupported statements and unsafe recommendations separately, report agreement before adjudication, and retain the original model output and prompt version. If external participants are recruited, the study will obtain the required institutional ethics approval before data collection.

E. Research Questions and Reproducibility Artifacts

The study evaluates four bounded research questions: *RQ1*, whether the selected rule set detects the labelled findings in S1–S6; *RQ2*, whether generated candidate paths satisfy the expected-path conditions; *RQ3*, whether the heuristic ranking agrees with the pre-specified reference ranking; and *RQ4*, whether LLM-assisted reports meet the reviewer rubric without unsupported or unsafe recommendations. The versioned scenario catalogue, ground-truth definitions, and validation script are maintained in the repository root under `evaluation-scenarios/`. For each executed run, the study will archive sanitised configuration snapshots, source and tool versions, normalised outputs, graph evidence, measurement records, and cleanup confirmation. Secrets, credentials, unredacted logs, and sensitive asset identifiers are excluded from the repository.

V. IMPLEMENTATION STATUS AND BASELINE DEPLOYMENT EVIDENCE

At the time of writing, the controlled AWS baseline and S5 private workload have been deployed in an isolated evaluation account in the `ap-south-2` region and are managed by Terraform. The deployment includes test-only S3 controls, a dedicated IAM role, an isolated VPC, and one private S5 instance with a metadata-only secret and synthetic S3 object. The resource identifiers, execution artifacts, and account-specific logs are retained outside the repository; only sanitised evidence is archived.

TABLE IV
VERIFIED CONTROLLED-BASELINE STATE

Component	Observed configuration verification
S3 controls	All four S3 public-access blocks enabled; AES-256 default encryption enabled; S5 uses one non-sensitive synthetic object.
IAM role	Dedicated role has no attached managed policies; its inline policy permits bucket listing/object read and `DescribeSecret`, but not secret-value retrieval.
Network boundary	VPC has no Internet gateway or external route; S5 uses a private subnet without automatic public IP assignment.
S5 workload	One private `t3.micro` instance; no public address; IMDSv2 required with hop limit one; encrypted root volume.
Security-group fixture	The S3 fixture has one `tcp/8443` ingress rule from `0.0.0.0/0`; the S5 group has no ingress or egress.

Read-only AWS API checks verified the state in Table IV, and a subsequent Terraform plan reported no pending resource changes. The baseline pass produced one medium-severity configuration finding for the security-group ingress rule that permits `0.0.0.0/0` as a source. It produced no candidate path: the required preconditions—an Internet gateway, subnet, and network interface attached to the group—were all absent. A separate temporary S1/S2 configuration pass disabled the bucket-level S3 public-access blocks and changed the test role to `s3:*` on the empty test bucket. The evaluator detected both high-severity configuration conditions and still produced no candidate path. The intended S1 public `GetObject` policy was rejected by an account-level S3 Block Public Access safeguard; no object, public policy, or public data was created. Terraform then restored all four bucket public-access blocks and the role's `ListBucket`/`GetObject`-only policy. For S4, a separate dedicated test trail was created in the disabled state; its `IsLogging` status was false, while its private log bucket retained all public-access blocks and AES-256 default encryption. No existing trail was stopped or modified, and the dedicated trail was never enabled. The S5 pass observed the private instance-profile relationship to the synthetic object and metadata-only secret, while confirming no public address, no ingress or egress, no Internet or NAT route, IMDSv2 with hop limit one, and no `GetSecretValue` permission. It therefore asserted no candidate path. These passes demonstrate bounded evidence collection and precondition rejection, not aggregate S1–S6 detection accuracy, candidate-path quality, timing, baseline-tool comparison, or LLM-report quality. S6 has not yet been executed.

VI. DISCUSSION AND LIMITATIONS

The proposed framework does not establish that a graph path is exploitable merely because related resources are connected. The threat model, evidence sources, path-validity rules, and score weights must be explicit. The controlled scenarios also do not represent all production environments, cloud providers, threat actors, or organisational policies. The initial scope is AWS only, and the in-memory graph design may not scale to large deployments without further engineering.

The LLM component adds cost, latency, privacy, and reliability concerns. It must not receive unapproved sensitive data, and its recommendations must not be executed automatically. These limitations motivate a staged evaluation in which each contribution is tied to an implementation artifact, metric, evidence source, and stated limitation.

VII. CONCLUSION

This paper presented CloudSentinel AI, a proposed framework for combining AWS configuration assessment, knowledge-graph modeling, candidate attack-path analysis, contextual prioritisation, and LLM-assisted reporting. The central contribution is a reproducible evaluation design that separates labelled finding detection, candidate-path validity, prioritisation, and explanation quality. Future work will integrate the read-only evidence collector into the product pipeline, complete the remaining rule implementations, execute the controlled scenario catalogue, archive raw evidence, and report bounded results without generalising beyond the evaluated configurations.

REFERENCES

[1] P. Mell and T. Grance, "The nist definition of cloud computing," National Institute of Standards and Technology, Tech. Rep. SP 800-145, 2011.

[2] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[3] A. Hogan, E. Blomqvist, M. Cochez, C. D'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. Labra Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, and A. Zimmermann, "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, 2021.

[4] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, 2002.

[5] S. Jha, O. Sheyner, and J. M. Wing, "Two formal analyses of attack graphs," in *Proceedings of the 15th IEEE Computer Security Foundations Workshop*, 2002.

[6] M. Elmiger, M. Lemoudden, N. Pitropakis, and W. J. Buchanan, "Start thinking in graphs: Using graphs to address critical attack paths in a microsoft cloud tenant," *International Journal of Information Security*, vol. 23, no. 1, pp. 467–485, 2024.

[7] T. van Ede, N. Khasuntsev, B. Steen, and A. Continella, "Detecting anomalous misconfigurations in aws identity and access management policies," in *Proceedings of the 2022 ACM Cloud Computing Security Workshop*, 2022, pp. 63–74.

[8] G. Rjoub, J. Bentahar, O. Abdel Wahab, R. Mizouni, A. Song, R. Cohen, H. Otrok, and A. Mourad, "A survey on explainable artificial intelligence for cybersecurity," *IEEE Transactions on Network and Service Management*, vol. 20, no. 4, pp. 5115–5140, 2023.

[9] H. Xu, S. Wang, N. Li, K. Wang, Y. Zhao, K. Chen, T. Yu, Y. Liu, and H. Wang, "Large language models for cyber security: A systematic literature review," 2024.